

Mini-projet 09 — classmethod, staticmethod, property

Module B · fin POO · Notions : @classmethod · @staticmethod · @property · 1er contact avec les décorateurs · Durée : 1 h 30 · Parcours Python — Ednir

Pourquoi ce mini-projet ?

Ta classe `Cible` sait déjà s'afficher et se comparer. Là on la rend **pro** avec trois outils : un **constructeur alternatif** (créer une `Cible` à partir d'une chaîne `"ip:port"`), une **fonction utilitaire** rangée dans la classe, et un **attribut calculé** (une info qui se déduit des données, sans être stockée). Bonus : c'est ta première rencontre avec les **décorateurs** (les `@truc`), qu'on creusera au MP10.

1. Rappel : c'est quoi un @décorateur ?

Un `@quelquechose` juste au-dessus d'une méthode, c'est un **modificateur** : il change la façon dont Python traite la méthode. Tu en as déjà vu un au MP07 : `@abstractmethod`. Ici tu en découvres trois autres, tous fournis par Python. Pas besoin de comprendre leur mécanique interne aujourd'hui (ce sera le MP10) — juste ce que chacun fait.

2. @classmethod — un constructeur alternatif notion clé

Une méthode de classe reçoit non pas `self` (un objet) mais `cls` (la **classe elle-même**). Usage le plus courant : fabriquer un objet **autrement** qu'avec le `__init__` normal.

```
class Cible:
    def __init__(self, ip, port):
        self.ip = ip
        self.port = int(port)

    @classmethod
    def depuis_texte(cls, texte):
        # cls = la classe Cible elle-même
        ip, port = texte.split(":")
        return cls(ip, port)  # cls(...) == Cible(...) : on construit un objet

c = Cible.depuis_texte("10.0.0.5:22")  # on appelle SUR la classe, pas un objet
print(c.adresse())  # 10.0.0.5:22
```

Pourquoi `cls` et pas `Cible` en dur ? Si une classe héritait de `Cible`, `cls` renverrait automatiquement la bonne sous-classe. C'est plus souple que d'écrire `Cible(...)` en dur. On appelle une classmethod **sur la classe** : `Cible.depuis_texte(...)`.

3. @staticmethod — une fonction utilitaire rangée dans la classe

Une méthode statique ne reçoit **ni `self` ni `cls`**. C'est une simple fonction, mais rangée dans la classe parce qu'elle est **en lien avec le sujet**. Utile pour une petite vérification qui n'a pas besoin d'un objet.

```
@staticmethod
def port_valide(port):
    # pas de self, pas de cls
    return 0 <= port <= 65535

Cible.port_valide(22)  # True — appelée sur la classe, sans créer d'objet
Cible.port_valide(99999)  # False
```

Quand l'utiliser : une fonction logiquement liée à la classe mais qui ne dépend d'aucun objet précis. Ici, « ce port est-il dans les bornes valides ? » n'a pas besoin d'une `Cible` existante pour répondre.

4. @property — un attribut calculé

Une propriété est une **méthode qu'on lit comme un attribut** : **sans les parenthèses**. Elle sert à exposer une info **calculée** à partir des données de l'objet, sans la stocker.

```
@property
def est_bien_connu(self):
    return self.port < 1024  # ports "bien connus" : 22, 80, 443...
```

```
c = Cible("10.0.0.5", 22)
print(c.est_bien_connu)  # True    - PAS de () ! on la lit comme un attribut
```

Le piège : avec `@property`, tu écris `c.est_bien_connu` et **surtout pas** `c.est_bien_connu()`. Si tu mets les parenthèses, tu obtiens une erreur (`bool is not callable`) car la propriété a déjà renvoyé un booléen.

5. Le récap des trois

Décorateur	1er paramètre	On appelle...	Sert à
<code>@classmethod</code>	<code>cls</code> (la classe)	<code>Cible.depuis_texte(...)</code>	constructeur alternatif
<code>@staticmethod</code>	rien de spécial	<code>Cible.port_valide(...)</code>	fonction utilitaire liée
<code>@property</code>	<code>self</code>	<code>c.est_bien_connu</code> (sans <code>()</code>)	attribut calculé

Ta mission

Objectif : enrichir ta classe `Cible` (celle du MP06) avec les trois décorateurs.

Repars de ta classe `Cible` et ajoute :

- `@classmethod depuis_texte(cls, texte)` → découpe `"ip:port"` et renvoie un objet via `cls(ip, port)`.
- `@staticmethod port_valide(port)` → renvoie `True` si `0 <= port <= 65535`, sinon `False`.
- `@property est_bien_connu(self)` → renvoie `True` si le port est `< 1024`.

Tests à faire passer :

```
c = Cible.depuis_texte("10.0.0.5:22")
print(c)                # -> Cible(10.0.0.5:22)
print(type(c.port))     # ->

print(Cible.port_valide(22))    # -> True
print(Cible.port_valide(99999)) # -> False

print(c.est_bien_connu)    # -> True    (PAS de parenthèses !)
d = Cible("10.0.0.5", 8080)
print(d.est_bien_connu)    # -> False
```

Bonus : dans `depuis_texte`, utilise ta `@staticmethod port_valide` pour **lever une exception** (ton `CibleInvalide` du MP05, ou un simple `ValueError`) si le port est hors bornes. Tu combines ainsi deux mini-projets.

✓ Critères de réussite

- `Cible.depuis_texte("10.0.0.5:22")` renvoie bien un objet `Cible` (port en `int`).
- `Cible.port_valide(...)` s'appelle sur la classe, sans créer d'objet.
- `c.est_bien_connu` se lit **sans** parenthèses et renvoie `True` / `False`.

Si tu bloques — indices progressifs

Étape	Indice
classmethod	Le décorateur <code>@classmethod</code> au-dessus, 1er paramètre <code>cls</code> , et à la fin <code>return cls(ip, port)</code> (pas <code>Cible(...)</code> en dur).
staticmethod	<code>@staticmethod</code> , pas de <code>self</code> ni <code>cls</code> dans les paramètres, juste <code>port</code> . Une seule ligne : <code>return 0 <= port <= 65535</code> .
property	<code>@property</code> , garde <code>self</code> , et surtout à l'usage : <code>c.est_bien_connu</code> sans <code>()</code> .
Bonus	Dans <code>depuis_texte</code> , après avoir converti le port : <code>if not cls.port_valide(int(port)): raise ...</code> .

Où ranger : `mini-projets/09-decorateurs-classe/cible_pro.py`.

Quand c'est prêt : `git add` → `commit` → `push`, puis « c'est poussé ». Ce sera la **fin du Module B** — on fera un bilan avant le Module C (les outils de puissance : décorateurs, générateurs, context managers).